

HARDNESS

P, NP-complete (NPC)

SAT: satisfiability of Boolean formulas

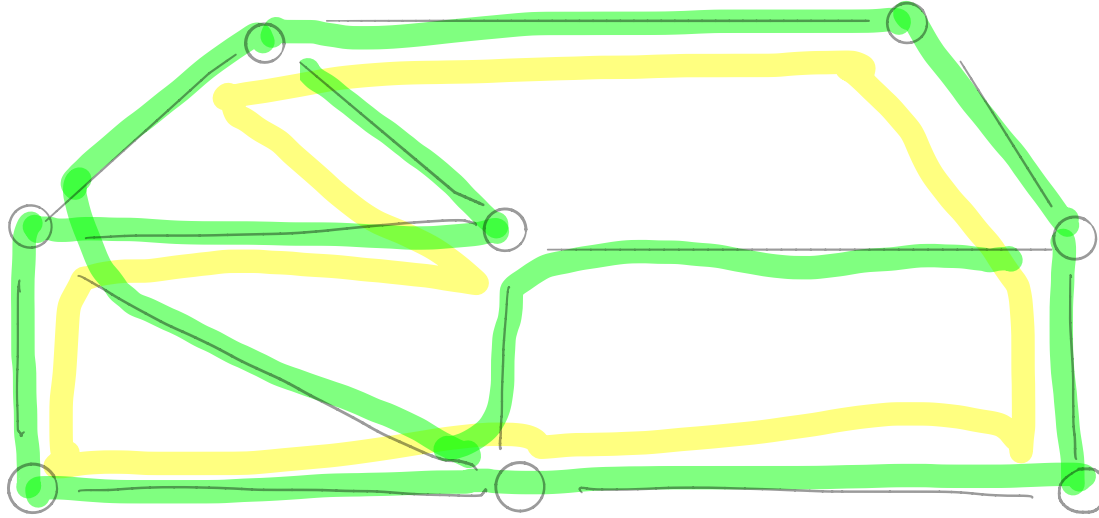
- N variables: $x_1, x_2, \dots, x_N \in \{0, 1\}$
- Boolean formula $F(x_1, x_2, \dots, x_N)$ = $(x_2 \vee \bar{x}_3 \vee x_8) \wedge (\bar{x}_1 \vee x_7 \vee x_8)$
CNF (conjunctive normal form) e.g. $\overset{T}{(x_2 \vee \bar{x}_3 \vee x_8)} \wedge (\bar{x}_1 \vee x_7 \vee x_8) \overset{T}{}$ (special form: 3-SAT)
- Establish if $\exists$ truth assignment $\gamma: \langle x_1, \dots, x_N \rangle \rightsquigarrow \{0, 1\}^N$
s.t. $F(\gamma(x_1, x_2, \dots, x_N)) = 1$ (TRUE) (0 means FALSE)

Th. Cook-Levin: $SAT \in NPC$

Every problem in NP reduces polynomially to SAT

it can be verified in polynomial time (poly-time certificate)
we do not know $P \stackrel{?}{=} NP$

How to deal with NP-hard / complete problems



Paths in a graph

- ↳ Eulerian path: all edges are traversed once : linear-time!
- ↳ Hamiltonian path: all nodes are traversed once : NPC!

KNAPSACK

Application example: we have to implement a file download manager on a server that has a block transfer of W bytes

- n files: size $w_1, w_2, \dots, w_n$
value $v_1, v_2, \dots, v_n$

(the higher, the better)
e.g. priority

- goal: transfer the max value of files staying within W bytes
requirement: files cannot be split

General formulation:

IN integer W (max capacity)
array $W[]$ of n sizes (positive integers)
array $V[]$ of n values

OUT subset $S \subseteq [n]$ that maximizes $\sum_{i \in S} V[i]$ (value)
under the condition $\sum_{i \in S} W[i] \leq W$ (S should fit into the knapsack)

We see how to get the optimal value $OPT = \sum_{i \in S^*} V[i]$

It's not difficult to find also S^* .

REMARK | Knapsack $\in$ NPC in its decision version $OPT \geq \alpha$

By restriction, it models PARTITION (how to split a multi-set of integers in two groups having same sum)

knapseack is a good illustrative problem for our study:

"Ok this problem is hard but, still, what can we do?"

- ✓ - exact computation for some special cases
 - exact computation: parameterized algorithms
 - close to optimal solution: approximate algorithms

BASELINE

Brute-force enumeration

```
best = 0, bestS = {}  
opt(1, {}, 0, 0)  
return best
```

$$\tilde{O}(2^n) \text{ where } \tilde{O}(f(n)) \equiv O(f(n) \cdot n^{O(1)})$$

we ignore poly factors

$OPT(i, S, \underset{w_i}{size}, \underset{v_i}{sum})$:

if $(i > n)$:
if $sum > best$:
 $best = sum$
 $bestS = S$

base
code

[

else:

binary partition

BT[i]=0
discard i

choose i
BT[i]=1

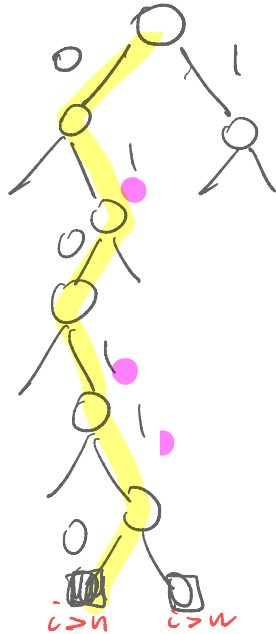
$OPT(i+1, S, \text{size}, \text{sum})$

if $size + w_i \leq W$:

$OPT(i+1, S \cup \{i\}, size + w_i, sum + v_i)$

OBS Implicitly...
Equivalent to generate
all binary sequences B
of length n , so that
 $S = \{i \in [n] : B[i] = 1\}$
such that $\sum_{i \in S} w_i \leq W$

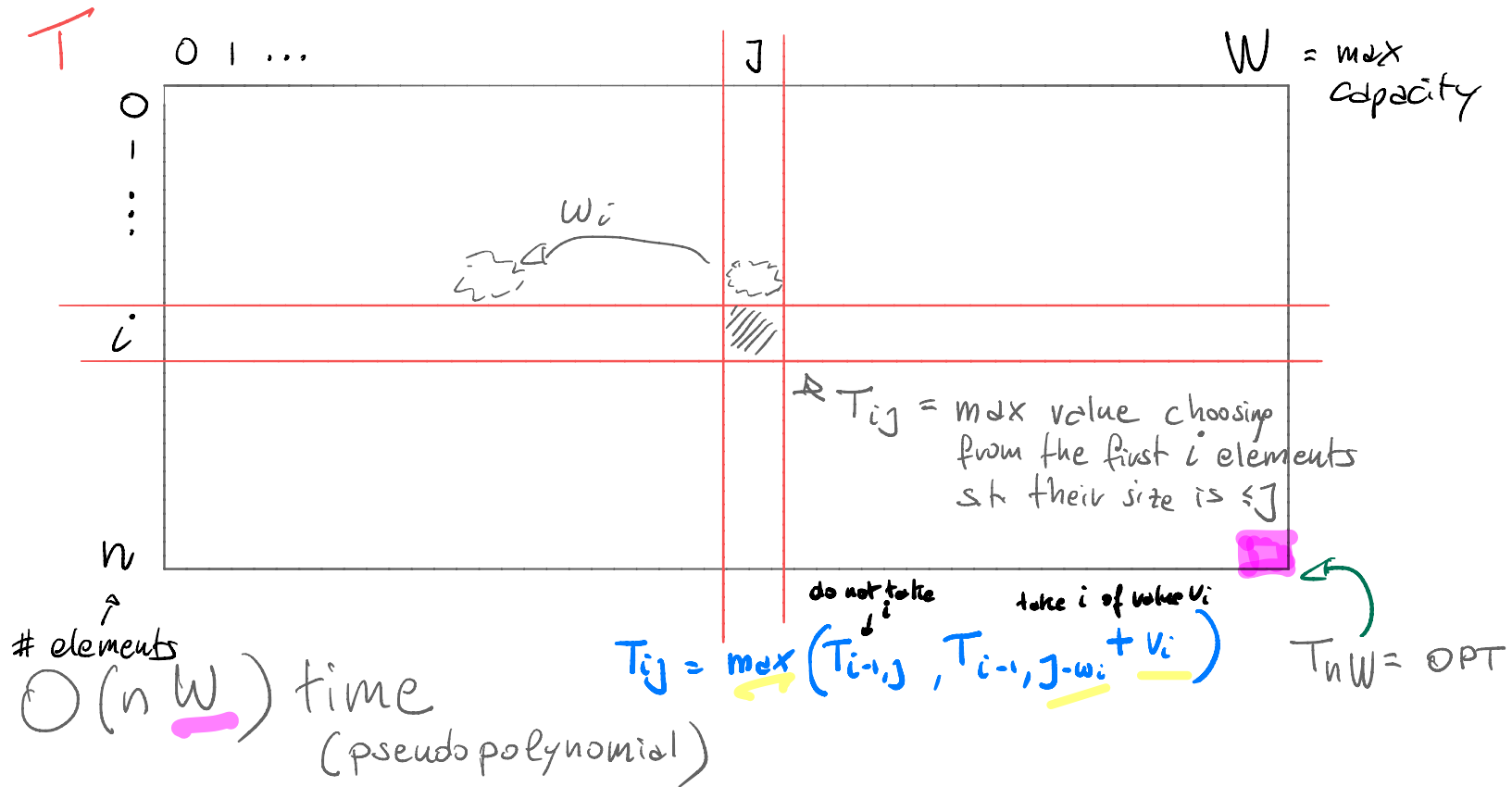
Corresponding recursion tree:

$$i \in I$$
$$\bar{c} = 2$$
$$\bar{C}_i = 3$$
$$C' \approx 4$$
$$c' = 5$$
$$C \geq 6$$


$$S = \{2, 4, 5\}$$

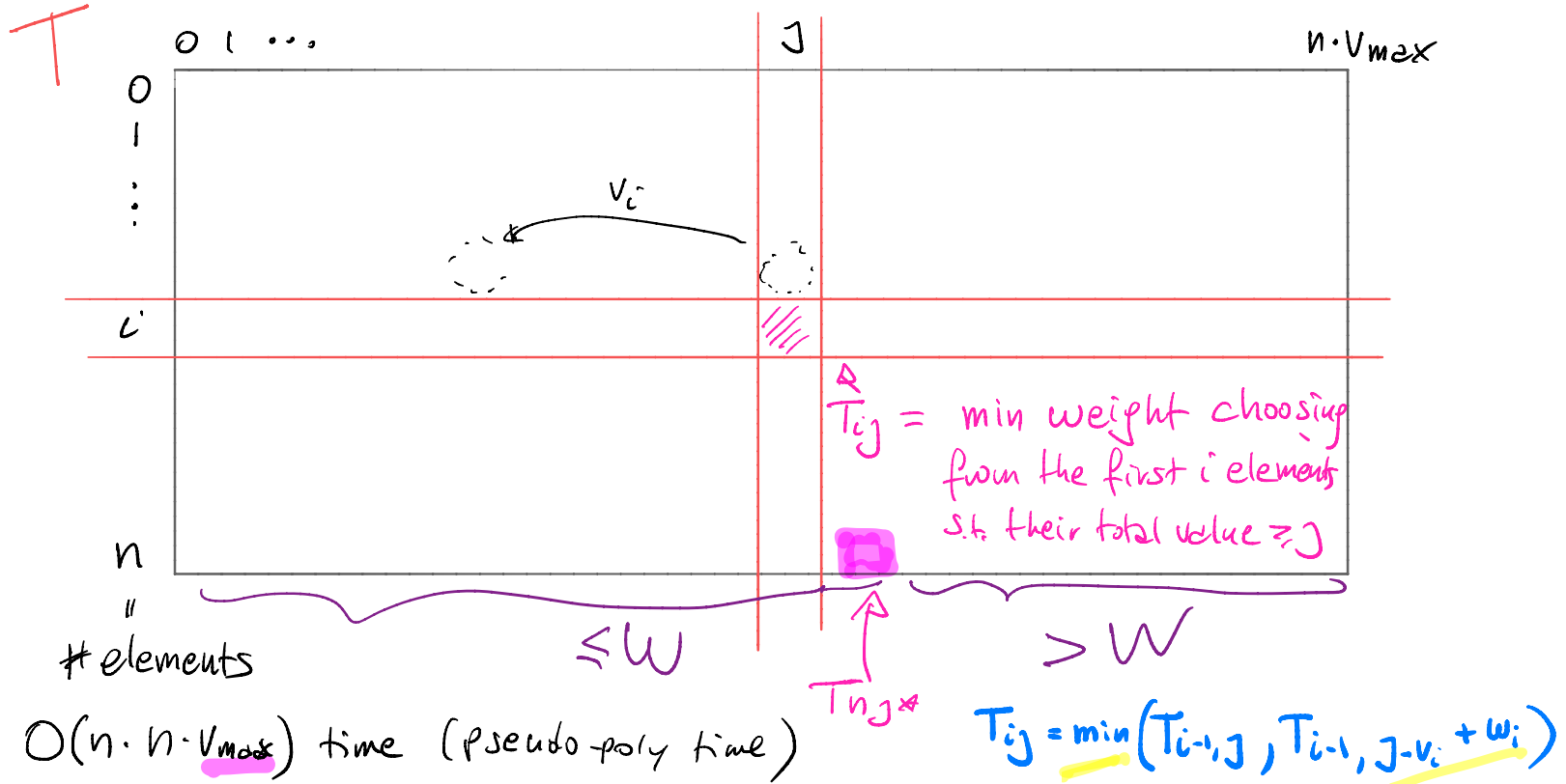
$B[i] = 0$

DYNAMIC PROGRAMMING: DP 1



DYNAMIC PROGRAMMING: DP2

$$V_{\max} = \max_{1 \leq i \leq n} V_i$$



EXACT SOLUTION.

Baseline	$\tilde{O}(2^n)$
DP 1	$O(nW)$
DP 2	$O(n^2 \cdot V_{\max})$



pseudo polynomial time

exponential time when W or $V_{\max}$ are very large

↳ APPROX ALG.
↳ PARAMETERIZED ALG.

COMBINATORIAL PROBLEM : several flavors

- ✓ • DECISION PROBLEM: does a solution exist? Yes/No
 - ↑
 - ✓ • SEARCH PROBLEM: give a solution
 - ↑
 - ✓ • OPTIMIZATION PROBLEM: give the "best" solution (according to a measure)
 - ↑
 - • COUNTING PROBLEM: the number of solutions (e.g. #P class)
 - ↑
 - ✓ • ENUMERATION PROBLEM: list all the solutions
 - ↑
- ↳ CLIQUE

We will see counting, using KNAPSACK
(sometimes is called # KNAPSACK)

0-1 KNAPSACK : no values, only weights (a.k.a. SUBSET SUM)

In this case, we want to count all possible choices $S \subseteq [n]$
s.t. $\sum_{i \in S} w_i \leq W$.

EXACT SOLUTION : based on DP 1 in $O(nW)$ time:

$$T_{ij} = \begin{cases} 1 & \text{if } i=0 \text{ or } j=0 \\ T_{i-1,j} + T_{i-1,j-w_{i-1}} & \text{o.w.} \end{cases}$$

can be large?

NO &

YES

choose i -th element?

it's 0 when $j < w_{i-1}$ (for the i -th element)

OUTPUT: T_{nW}

HANDS-ON : UNIFORM SAMPLING using DP 1

return one feasible choice S uniformly at random $\equiv$ pr. is $\frac{1}{T_{nW}}$
 $\sum_{i \in S} w_i \leq W$

Lesson learned : if we know how to count, we can also sample using dynamic programming

We follow the idea of using the conditional probabilities that have been implicitly computed with the DP.

CAVEAT: when solution sets are partially overlapping, this counting method does not work as it can overcount.

Hence, sampling works when the corresponding dynamic programming scheme does not overlap solutions.

What if W is large? We cannot afford exact counting...

... let's do approximate counting
(next class)

What is the class corresponding to NP for counting?

NP = { problems Π : they have a deterministic poly-time Certificate }

binding sequences

Certificate $V(x, y)$ = deterministic poly-time verifier s.t. $\forall$ input x

solution $\cdot x \in \Pi \Rightarrow \exists y : V(x, y) = \text{YES}$, $|y| = \text{poly}(|x|)$

NOT solution $\cdot x \notin \Pi \Rightarrow \forall y : V(x, y) = \text{NO}$

$\#P = \{ \text{problem } \Pi : \Pi \text{ has poly-time certificate } V(x, y) \text{ and output is } \#y \text{ s.t. } V(x, y) = \text{YES} \}$

i.e. we count how many solutions

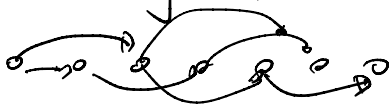
#P-complete problems are the "most" difficult problems in #P
(as NP-complete for NP)

Reduction is called "parsimonious" (preserves polynomially
the number of solutions)

NO SURPRISE: NPC $\Pi \Rightarrow$ #P-complete $\# \Pi$

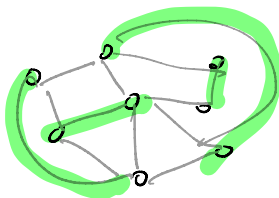
SURPRISING: some P $\Pi \Rightarrow$ #P-complete

Eg. topological sorting of a DAG



counting them is
#P-complete. $\triangleright$

Eg. perfect matching



use poly-time
flow alg.

counting them is
#P-complete

E.g. regular expression r

it represents a set of strings

counting the number of
represented strings is #P-complete